

How to Approximate A Set Without Knowing Its Size In Advance

241880161 仇浩嘉 241880136 任春驿 241880070 周文成

2026 年 7 月 27 日

目录

1	论文解决了什么问题	2
2	这个问题为什么重要	2
3	相关研究	2
4	主要定理与贡献	4
5	下界证明思路	5
6	上界构造细节	8
7	总结与展望	13

1 论文解决了什么问题

1.1 经典问题

Approximate membership data structures (近似成员查询数据结构): 给定一个可以在线插入元素的集合 S (大小为 n), 支持成员查询, 并且需要满足以下两个条件:

- **无假阴性**: $\forall x \in S$, 查询总是返回 Yes。
- **假阳性率最多 ε** : $\forall x \notin S$, 查询返回 Yes 的概率最多达到给定的 ε 。

当集合大小 n 事先已知时, 空间下界为 $\Theta(n \log(1/\varepsilon))$ 比特, 即每个元素仅需 $\Theta(\log(1/\varepsilon))$ 比特。这一结论是信息论的基本结果, 且已有接近此下界的高效数据结构 (如 Bloom Filter 及其改进)。

1.2 本文研究的变体

已有的近似成员数据结构需要集合 S 的大小 n 已知。该论文研究的是 n **未知** 的情形: 插入以流式在线进行, 数据结构无法预知最终会插入多少个元素, 必须自适应地扩容, 同时维持空间使用尽可能小。论文从理论基础角度出发, 首次给出了该变体下的空间下界和匹配的上界构造。

2 这个问题为什么重要

现代计算环境本质上是流式计算:

- **数据随机**: 无法预知会有多少数据量到达。
- **突发流量**: 即使有历史统计平均值作参考, 也难免出现数据量激增导致 n 的值翻倍的情况。
- 如果数据结构严格要求预先知道 n 的大小, 在流量激增的时候要么拒绝服务, 要么假阳性率失控。

在大数据流场景 (如网络监控、数据库缓存、分布式系统) 中, 集合大小往往无法提前预知。本文工作解释了为什么简单扩展 Bloom Filter 会浪费空间, 并为设计“空间自适应”的近似成员结构提供了理论指导。

3 相关研究

3.1 Bloom Filters (布隆过滤器)

- Bloom Filter 是一种经典的数据结构, 天然支持动态插入。其空间开销是信息论下界 $n \log(1/\varepsilon)$ 的 $\log(e) \approx 1.44$ 倍。

- 标准 Bloom Filter 不支持从集合 S 中删除元素，因为将任意位重置为 0 可能会导致假阴性。
- 为支持删除，可以使用 Counting Bloom Filter（计数布隆过滤器），每个 bit 替换为一个计数器。但这会使空间使用增加 $\Omega(\log \log n)$ 倍。这种删除支持是有条件的：只有在不删除假阳性元素时，数据结构才能正确工作，但根据定义，无法完全避免对假阳性元素的删除。
- Cohen 和 Matias 提出了一种方法，将空间额外开销降低至 $O(n)$ 位，并将近似成员查询推广到了多重集的近似重数。

3.2 基于字典的近似成员查询

- 早在 1978 年，Carter 等人就提出了一种能达到类似结果的技术。他们观察到，维护一个多重集 $h(S)$ （其中 $h: [u] \rightarrow [n/\varepsilon]$ 是一个通用哈希函数），如果集合 $h(S)$ 的存储空间接近信息论下界 $\log \binom{n/\varepsilon+n}{n}$ 位，那么整体解决方案的空间消耗仅为 $n \log(1/\varepsilon) + O(n)$ 位。
- 如果不需要支持删除操作，只需存储不同的哈希值集合 $h(S)$ 即可。这种动态集合可以被简洁地存储，且所有操作以高概率在 $O(1)$ 时间内完成。如果支持动态多重集（即支持删除），可以通过转化为标准成员查询问题来实现，代价是更新操作需要更大的摊销时间。

3.3 在线/离线空间需求的差异

- 在静态情况下，已经证明能够逼近 $n \log(1/\varepsilon)$ 的空间下界。例如，Dietzfelbinger 和 Pagh 证明了在查询时间为 $\omega(\log(1/\varepsilon))$ 且 ε 为 2 的整数幂时，空间开销可以控制在 $o(n)$ 项以内；Porat 在常数查询时间的情况下实现了相同的结果；随后 Bellazougui 和 Venturini 进一步消除了对 ε 的限制，同时维持了常数查询时间。
- 然而，Lovett 和 Porat 证明了这些静态情况下的结果无法直接扩展到允许动态更新的场景：需要 $\Omega(n/\log(1/\varepsilon))$ 位的额外空间开销。即使在整个插入序列结束前不进行任何查询，该下界依然成立。这意味着，对于以数据流形式给出的键集，仅使用接近静态大小下界的空间来构建近似成员数据结构是不够的。

3.4 动态空间使用

- 当要求空间必须依赖于集合的当前大小时，上界要求更加严格。
- 文献中现有的相关技术通常会导致 $\Omega(\log n)$ 或 $\Omega(\log(1/\varepsilon))$ 的查询时间，以及 $\Omega(n \log n)$ 位的高空间开销。
- 这些数据结构通常采用维护多个近似成员数据结构并逐一查询的策略。如果采用几何级数递增的容量，就需要 $\Omega(\log n)$ 个这样的数据结构。每一个结构对应的假阳性率 $\varepsilon_1, \varepsilon_2, \dots$ 之和必须收敛于目标 ε 。例如在已有的 Scalable Bloom Filter 中，让 ε_i 随 i 呈几何级数递减，这会导致 $\varepsilon_i = n^{-\Omega(i)}$ ，从而产生 $\Omega(n \log n)$ 的空间使用量——渐近上远非最优。

3.5 动态完美哈希与检索

- 在静态情况下，一种实现近似成员查询的方法是存储一个完美哈希函数，将键单射地映射到 $\{1, \dots, n\}$ ，然后在数组中根据该完美哈希函数为每个键存储 $\log(1/\varepsilon)$ 位的签名。
- 更一般地，动态检索数据结构（如 Bloomier filters）也可以用来构建动态近似成员数据结构。研究表明，在线设置下，这两个问题都需要 $\Theta(n \log \log n)$ 的空间。然而，这些上界具有固定的空间使用量（在常数因子内），因此无法实现本文所获得的那种随集合大小渐进最优的空间消耗。

4 主要定理与贡献

论文在集合大小 n 事先未知的设定下，给出了近似成员查询问题的空间下界和上界，并给出两种有效构造。

4.1 定理 1.1（下界）

任意针对未知大小集合 S 、假阳性率 ε 的近似成员数据结构，在某 $n > u^\delta$ 次插入后，必须使用至少

$$(1 - o(1))n \log(1/\varepsilon) + \Omega(n \log \log n)$$

位的空间。

- 第一项 $(1 - o(1))n \log(1/\varepsilon)$ 是已知 n 时就已经需要的（与传统下界一致）。
- 第二项 $\Omega(n \log \log n)$ 是”不知道 n ”所付出的额外代价。

注：条件 $n > u^\delta$ 的含义是： $\delta \in (0, 1)$ 为任意小常数， n 与全集大小 u 之间呈多项式关系（例如 $n > \sqrt{u}$ 或 $n > u^{0.01}$ ）。这排除了 n 极小（如 $n = O(\log u)$ ）的退化情况。此外，在实际场景中，假阳性率常取一个不太小的常数（如 $\varepsilon = 1/10$ ），此时下界意味着平均每元素必须使用 $\Omega(\log \log n)$ 比特——这在传统已知 n 的设定下是常数级的。

4.2 定理 1.2（上界）

论文给出两种匹配下界的构造，以及构造 2 的去摊销优化版本，具体信息如下。

属性	构造 1（热身）	构造 2（精妙）	构造 2（去摊销）
空间	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$O(n \log(1/\varepsilon) + n \log \log n)$
假阳性率	$\leq \varepsilon$	$\leq \varepsilon + u^{-c}$	$\leq \varepsilon + u^{-c}$
假阴性	无	无	无
插入时间	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
查询时间	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
是否需要已知 n	否	否	否

4.3 与 Bloom Filter 的对比

为更直观地展示本文构造的优势，下面给出与经典 Bloom Filter 的横向对比：

	Bloom Filter	构造 1	构造 2	构造 2 去摊销
空间（每元素）	$1.44 \log(1/\varepsilon)$	$(1 + o(1)) \log(1/\varepsilon) + O(\log \log n)$	同构造 1	$O(\log(1/\varepsilon) + \log \log n)$
已知 n ?	是	否	否	否
查询时间	$O(\log(1/\varepsilon))$	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
插入时间	$O(\log(1/\varepsilon))$	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
空间紧致	否 ($1.44\times$)	是	是	否（常数损失）
结构数量	1	$\log n$	1	1
依赖的哈希	k 个独立哈希	通用哈希	成对独立	成对独立
支持删除	否	否	可（扩展）	可（扩展）

5 下界证明思路

本章详细阐述论文第三章中空间下界的完整证明链路。证明的核心技术是**压缩论证**（compression argument）：将数据结构用作一种编码工具，利用信息论下界（无损编码所需的最小比特数）反推出数据结构的空間下界。

5.1 从随机结构到确定性结构（引理 3.2）

首先定义几个关键概念：

- **随机比特串** r ：数据结构内部所有哈希、随机选择所依赖的只读随机源，其存储空间不计入数据结构开销。
- **确定性结构** B_r ：将随机源 r 固定后，整个插入、查询流程不再存在任何随机行为，输入完全决定输出。
- **判定超集** $\hat{S}_r$ ：对于插入序列 S ，所有被 B_r 查询返回“存在”的元素构成的集合。无假阴性保证 $S \subseteq \hat{S}_r$ 。
- **归一化测度** $\mu(\hat{S}_r) = |\hat{S}_r|/u$ ：代表判定超集在全集 U 中的占比，等价于随机抽取一个元素被误判为 Yes 的概率。
- **合法序列集合** $S_{r,\varepsilon} = \{S \in U^n \mid \mu(\hat{S}_r) \leq 4\varepsilon\}$ ：固定随机种子 r 后，判定超集规模可控的那些插入序列。

引理 3.2：存在随机串 r^* ，使得 $|S_{r^*,\varepsilon}| \geq u^n/2$ 。即至少一半的插入序列在固定 r^* 后假阳性测度不超过 4ε 。

证明过程：

1. 对任意长度为 n 的插入序列 S ，期望假阳性测度满足：

$$\mathbb{E}_{r \leftarrow \{0,1\}^*} [\mu(\hat{S}_r)] \leq \varepsilon + \frac{n}{u}$$

由于论文限定 $n \leq \varepsilon u$ ，有 $n/u \leq \varepsilon$ ，因此上式右端 $\leq 2\varepsilon$ 。

2. 使用**马尔可夫不等式**： $\mu(\hat{S}_r) \geq 0$ 为非负随机变量，取阈值 $a = 4\varepsilon$ ：

$$\Pr [\mu(\hat{S}_r) \geq 4\varepsilon] \leq \frac{\mathbb{E}[\mu(\hat{S}_r)]}{4\varepsilon} \leq \frac{2\varepsilon}{4\varepsilon} = \frac{1}{2}$$

3. 全空间 U^n 共有 u^n 条插入序列。对任意 r ，落入 $S_{r,\varepsilon}$ 的序列比例 $\geq 1/2$ 。因此，必存在某个 r^* 使得 $\geq u^n/2$ 条序列满足 $\mu \leq 4\varepsilon$ 。

5.2 指数分段与引理 3.3 (抽屉原理)

获得确定性结构 B_{r^*} 后，对任意合法序列 S 进行指数分段：

- 定义分段基数 $\gamma \geq 2$ (可调参数，控制分段长度的增长速度)。
- 第 i 段子序列 C_i 的长度严格为 γ^i 。
- 前缀 S_i ：前 i 段拼接，长度 $n_i = \sum_{j=1}^i \gamma^j$ 。
- 由于无假阴性，判定超集单调不减： $\hat{S}_1 \subseteq \hat{S}_2 \subseteq \dots$ ，因此测度也单调不减： $0 \leq \mu(\hat{S}_1) \leq \dots \leq 4\varepsilon$ 。

引理 3.3 (抽屉原理)：对任意 $S \in \mathcal{S}$ ，存在下标 i 满足前缀长度 $n_i \in [\alpha n, n]$ ，且超集测度增量：

$$\mu(\hat{S}_i) - \mu(\hat{S}_{i-1}) \leq \frac{4\varepsilon}{\log_\gamma(1/\alpha) - 2}$$

即在整个增长区间中，必有一段测度增量不超过平均值。

5.3 引理 3.4 (切尔诺夫界控制重叠元素)

定义 $k = |C_i \cap \hat{S}_{i-1}|$ ，即 C_i 中在插入前就已被判为 Yes 的元素数 (重叠元素)。

引理 3.4：满足 $k(S) \leq 9\varepsilon|C_i|$ 的序列至少占 $u^n/3$ 。

证明使用了**切尔诺夫界**：假设序列逐段均匀独立从全集 U 采样， C_i 均匀随机， $\mathbb{E}[k] = |C_i| \cdot \mu(\hat{S}_{i-1}) \leq 4\varepsilon|C_i|$ 。取 $(1 + \delta) \cdot 4\varepsilon = 9\varepsilon$ 得常数 δ ，切尔诺夫界给出尾概率 $\Pr[k \geq 9\varepsilon|C_i|] \leq \exp(-|C_i|)$ 。由于 $|C_i| = n^{\Omega(1)}$ ，该尾概率指数级小。对所有分段使用**联合界**，结合引理 3.2 的 $|\mathcal{S}| \geq u^n/2$ ，最终满足条件的序列 $\geq u^n/3$ 。

5.4 四段式无损编码与压缩论证

对于任意一条合法序列 S ，构造一个四段编码。解码器仅依靠该编码串就能完整还原 S ：

1. 下标 i ： $\log \log n$ 比特。
2. C_i 以外的全部元素：完整存储，开销 $(n - c_i) \log u$ 比特 ($c_i = |C_i|$)。
3. 完整数据结构内部状态： b_i 比特（含公共随机源 r^* ）。解码器结合前两段可以获得 S_{i-1} 的全部内容和 C_{i+1} 之后的所有元素，加上 b_i 位状态码，可以重建 B_{r^*} 在处理 C_i 前后的内部状态，从而还原 $\hat{S}_{i-1}$ 和 $\hat{S}_i$ 两个超集。
4. C_i 的相对压缩编码：分为两类——
 - k 个重叠元素（落在旧超集 $\hat{S}_{i-1}$ 内）：编码开销 $k \cdot \log(\mu(\hat{S}_i) \cdot u) + O(1)$
 - $c_i - k$ 个新增元素（落在 $\hat{S}_i \setminus \hat{S}_{i-1}$ 内）：编码开销 $(c_i - k) \cdot \log(\Delta\mu \cdot u) + O(1)$

总编码长度：

$$\text{TotalLen} = \log \log n + (n - c_i) \log u + b_i + (c_i - k) \log(\Delta\mu \cdot u) + k \log(\mu(\hat{S}_i)u) + O(c_i)$$

5.5 化简与矛盾不等式

将 $\log(\mu u)$ 拆分为 $\log u + \log \mu$ ，提取公共 $\log u$ 项：

$$\text{TotalLen} = b_i + \log \log n + n \log u + (c_i - k) \log \Delta\mu + k \log \mu(\hat{S}_i) + O(c_i)$$

代入三个引理的约束：

- 引理 3.3: $\log \Delta\mu \leq \log \varepsilon - \log \log_\gamma(1/\alpha) + O(1)$
- $\mu(\hat{S}_i) \leq 4\varepsilon \leq \varepsilon$ ，故 $\log \mu(\hat{S}_i) \leq \log \varepsilon$
- 引理 3.4: $k \leq 9\varepsilon c_i$

合并后得到：

$$\text{TotalLen} \leq b_i + \log \log n + n \log u + c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1))$$

信息论下界要求：能够区分 $\geq u^n/3$ 条不同序列的编码长度必须满足：

$$\text{TotalLen} \geq \log(u^n/3) = n \log u - O(1)$$

令编码上界 $\geq$ 信息论下界，消去 $n \log u$ 后移项，得到单段空间下界：

$$b_i \geq c_i \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - O(1) \right)$$

5.6 从 b_i 到 β : 完成定理 3.1

建立 c_i 与 n_i 的数量关系 (等比数列求和):

$$n_i = \gamma \frac{\gamma^i - 1}{\gamma - 1} = c_i \cdot \frac{\gamma - 1/\gamma^{i-1}}{\gamma - 1} \implies c_i = n_i \cdot \frac{\gamma - 1}{\gamma} \cdot (1 + o(1))$$

即 $c_i = \Theta(n_i)$ 。

代入反证假设 $b_i \leq \beta n_i$, 约去 n_i 得到:

$$\beta \geq \left(1 - \frac{1}{\gamma}\right) \cdot \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - \Theta(1)\right)$$

5.7 参数赋值导出主定理 1.1

- 取 $\alpha = 1$ (退化到已知规模): 令 $\gamma \rightarrow \infty$, 得 $\beta \geq (1 - o(1)) \log(1/\varepsilon)$ ——与传统下界一致, 验证了定理的自洽性。
- 取 $\alpha = 1/\sqrt{n}$, $\gamma = 2^{(\log n)^\eta}$ ($\eta \in (0, 1)$):

$$\log_\gamma \frac{1}{\alpha} = \log_{2^{(\log n)^\eta}} \sqrt{n} = \frac{\frac{1}{2} \log n}{(\log n)^\eta} = \frac{1}{2} (\log n)^{1-\eta} = \Theta(\log \log n)$$

回代总空间 $S = \beta n$:

$$S \geq (1 - o(1)) n \log \frac{1}{\varepsilon} + \Omega(n \log \log n)$$

5.8 下界证明链路小结

完整证明由六步组成:

1. 原始随机近似结构 $\rightarrow$ 马尔可夫不等式 (引理 3.2) $\rightarrow$ 固定 r^* 转为确定性结构;
2. 对插入序列指数分段 $C_i, S_i \rightarrow$ 抽屉原理 (引理 3.3): 单段超集增量可控;
3. 均匀采样 + 切尔诺夫界 (引理 3.4): 绝大多数分段重叠元素数量可控;
4. 构造四段无损编码 $\rightarrow$ 信息论编码下界建立矛盾不等式;
5. 化简得到单段 b_i 下界 $\rightarrow$ 结合 c_i, n_i 等比关系证明定理 3.1;
6. 代入两组参数 $\rightarrow$ 分别得到已知/未知规模空间下界 (定理 1.1)。

6 上界构造细节

6.1 预备知识

6.1.1 单位代价 Word RAM 模型

全集大小为 u , 单个元素占用一个字, 字长 $w = \lceil \log u \rceil$ 比特。在该模型中, 元素的加减、按位布尔运算、任意位数的移位、整数乘法均为单位耗时。本文将作为数据结构上下界分析

的统一基准模型；下界证明不关心操作时间，只在意存储空间比特数。

6.1.2 k 重独立哈希函数族

哈希函数族 $H : U \rightarrow V$ 是 k -wise independent 的，若任取 k 个互不相同的元素 $x_1, \dots, x_k$ 和任意目标取值 $y_1, \dots, y_k$ ，有：

$$\Pr_{h \leftarrow H}[h(x_1) = y_1 \wedge \dots \wedge h(x_k) = y_k] = \frac{1}{|V|^k}$$

其弱化版本 k -wise δ -dependent 允许与均匀分布的统计距离不超过 δ 。论文在构造 2 中仅需 pairwise independent ($k = 2$)，即可在碰撞分析和假阳性界证明中获得足够的随机性保证，同时计算高效（常数时间，仅需 $O(\log u)$ 位随机种子）。

6.1.3 基础概率工具

- **马尔可夫不等式**：设非负随机变量 $X \geq 0$ ，对任意 $a > 0$ ， $\Pr[X \geq a] \leq \mathbb{E}[X]/a$ 。在引理 3.2 中用于筛选优质随机种子。
- **切尔诺夫界**：独立伯努利随机变量之和偏离期望的概率呈指数衰减，放缩精度远高于马尔可夫不等式。在引理 3.4 中用于控制分段重叠元素数量。
- **联合界**： $\Pr[\bigcup_i A_i] \leq \sum_i \Pr[A_i]$ 。在构造 1 中用于多子过滤器总假阳性控制 ($\sum \varepsilon_i \leq \varepsilon$)，在引理 3.4 中用于所有分段同时可控的概率界。
- **无损编码信息论下界**：若存在 M 条互不相同的序列，采用无损前缀编码存储所有序列，则至少需要 $\log_2 M$ 比特。这是压缩论证的理论基础。

6.2 构造 1：几何增长数据结构（热身）

6.2.1 核心思想

将插入序列按指数分块，第 i 块包含 2^i 个元素。为每个子序列 i 分配一个动态近似成员数据结构 B_i ，其假阳性率 $\varepsilon_i = \Theta(\varepsilon/i^2)$ 。查询时依次询问所有 B_i ，任一返回 Yes 则判定存在。

6.2.2 B_i 的内部设计

每个 B_i 基于 dictionary-based approximate membership 方法构建：

- 选择哈希函数 $h : U \rightarrow [m]$ ，其中 $m \approx 2^i/\varepsilon_i$ 。
- 对每个插入元素 x ，计算 $h(x)$ ，然后把 $h(x)$ 存入一个精确的动态字典（Raman-Rao 动态字典 [RR03]）。
- 查询时：计算 $h(x)$ ，检查它是否在字典里。若在则返回 Yes。

Raman-Rao 动态字典 [RR03] 支持期望摊还 $O(1)$ 的插入、最坏 $O(1)$ 的查询，空间接近信息论下界 $((1+o(1)) \cdot n \cdot \log(m/n))$ 比特。在近似成员场景中，空间约为 $(1+o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ 比特。

6.2.3 整体框架与操作

- **插入**：假设将要插入的是第 m 个元素，通过 $2^i - 2 < m \leq 2^{i+1} - 2$ 确定应插入哪个 B_i 。若是该块的第一个元素，则新建 B_i ；否则直接插入。绝大多数插入是直接插入，新建 B_i 的成本被摊销——总体期望摊还 $O(1)$ 。
- **查询**：对当前所有已存在的 $B_1, \dots, B_k$ 逐个调用成员查询。任一返回 Yes 则返回 Yes，全部返回 No 才返回 No。由于 $k \approx \log n$ ，总查询时间为 $O(\log n)$ 。
- **假阳性控制**：由联合界，总假阳性率 $\leq \sum_{i=1}^{\infty} \varepsilon_i = \Theta(\varepsilon \cdot \pi^2/6) \approx \varepsilon$ 。

6.2.4 空间复杂度分析

单个 B_i 空间为 $(1+o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ 。代入 $\varepsilon_i = \Theta(\varepsilon/i^2)$ ：

$$\log(1/\varepsilon_i) = \log(1/\varepsilon) + 2\log i + O(1)$$

由 $i \leq \log n$ ，取 $\max_i \{\log(1/\varepsilon_i)\} = \log(1/\varepsilon) + O(\log \log n)$ 。总空间：

$$\sum_i (1+o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i) \leq (1+o(1)) \cdot n \cdot \max_i \{\log(1/\varepsilon_i)\} = (1+o(1))n \log(1/\varepsilon) + O(n \log \log n)$$

匹配下界。

6.2.5 优缺点

优点：实现简单直观，直观展示了“分块 + 指数衰减假阳性”的基本思想。

缺点：查询时间随 n 对数增长 ($O(\log n)$)，因为不知道元素属于哪个子序列，需遍历所有 B_i 。

6.3 构造 2：常数时间操作

6.3.1 总体目标

在构造 1 的基础上，进一步实现查询最坏情况常数时间，同时保持空间接近最优，并支持插入的期望摊销常数时间。最终可扩展到插入最坏情况常数时间。

6.3.2 核心设计思想

与构造 1 多结构并存的思路截然不同，构造 2 的核心理念是：

- 任意时刻**只维护一个动态字典** D_i （而非多个 B_i 并存）。

- 使用 pairwise independent 哈希函数 $h : U \rightarrow \{0,1\}^\ell$, 其中 $\ell \geq \lceil \log(1/\varepsilon) \rceil + \log u + 2$ 。
- 通过**签名逐步扩展 + 受控分支**, 将早期元素的影响控制在常数级别。
- 结合 **bucketing** (分桶) 实现空间最优。

6.3.3 哈希函数设置与分块逻辑

从成对独立哈希族 $\mathcal{H}$ 中采样 $h : U \rightarrow \{0,1\}^\ell$ 。对每个阶段 i , 从 h 的输出比特流中截取:

- $h_i(x)$ (签名/键): $h(x)$ 的最左 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$ 位, 作为字典键。
- $g_i(x)$ (辅助标签): $h(x)$ 的接下来 $r = \lceil \log \log u \rceil$ 位, 不足时以 $\perp$ 填充。

同样按 2^i 分块。任意时刻只存在一个活跃字典 D_i 。

6.3.4 插入操作 (两种模式)

Mode 1 (普通插入): 当前元素不是子序列的第一个元素。直接将 $(h_i(x), g_i(x))$ 以 $h_i(x)$ 为键存入当前字典 D_i 。时间 $O(1)$ 。

Mode 2 (阶段过渡): 当前元素是第 i 个子序列的第一个元素。执行字典切换:

1. 初始化新字典 D_i : 容量 2^{i+2} 项, 每项 $\ell_i + r$ 位。
2. 枚举 D_{i-1} 中所有记录 $(y, \alpha_1 \cdots \alpha_r)$ 进行迁移:
 - 若 $\alpha_1 \neq \perp$ (还有可用辅助位): 插入 $(y\alpha_1, \alpha_2 \cdots \alpha_r \perp)$ ——扩展一位签名。
 - 若 $\alpha_1 = \perp$ (辅助位已耗尽): 分支, 插入 $(y0, \alpha)$ 和 $(y1, \alpha)$ ——覆盖两种可能。
3. 释放 D_{i-1} 。然后按 Mode 1 将当前元素插入 D_i 。

6.3.5 正确性分析

字典容量不溢出: 将条目按来源分为两类——早期子序列 $(S_1, \dots, S_{i-r})$ 的标签已耗尽, 每个 S_j 元素贡献 2^{i-j-r} 个条目; 近期子序列 $(S_{i-r+1}, \dots, S_i)$ 的标签尚有剩余, 每元素贡献恰好 1 条目。总条目数:

$$|D_i| = \sum_{j=1}^{i-r} 2^{i-r-1} + \sum_{j=i-r+1}^i 2^j = (i-r) \cdot 2^{i-r-1} + 2^{i+1} - 2^{i-r+1} \leq 2^{i+2}$$

字典不会溢出。

无假阴性: 若 $\alpha_1 = \perp$ 则同时保留 $y0$ 和 $y1$, 覆盖了 $h_i(x)$ 所有可能的前缀。因此 x 在任意后续阶段 i 都至少有 $(h_i(x), \cdot)$ 在 D_i 中, 查询一定命中。

假阳性率 $\leq \varepsilon + u^{-c}$: 字典 D_i 中最多 2^{i+2} 个键, 由 pairwise independent 性质, $h_i(x)$ 与任一已有键碰撞的概率 $\leq 2^{i+2} \cdot 2^{-\ell_i}$ 。代入 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$, 碰撞概率 $\leq \varepsilon$ 。底层字典可

能以概率 u^{-c} 失败，通过将前 u^δ 个元素合并处理并使用联合界，可将失败概率控制在 u^{-c} 以内。因此假阳性率 $\leq \varepsilon + u^{-c}$ 。

6.3.6 查询操作

当前处于阶段 i 时，仅对当前字典 D_i 执行一次成员查询：用 $h_i(x)$ 作为键查找。命中则返回 Yes，否则返回 No。查询时间为**最坏情况** $O(1)$ （高概率）。

6.3.7 空间复杂度：桶化技术

基本构造中，从 D_i 过渡到 D_{i+1} 时两者需同时存在，空间暂时翻倍。为解决该问题，使用标准桶化技术：

- 用哈希将元素分摊到 $u^{\delta/2}$ 个桶中（ δ 为任意小正常数）。
- 各桶的数据结构按字交错存储（interleaved word-wise），确保过渡操作至多在一个桶中同时进行。
- 额外空间只与单个桶大小成比例，而非总元素数。

分析单个桶内的空间：设桶内有 n' 个元素（ $n' > u^\delta$ ），当前处于第 i 个子序列期间。底层 Raman-Rao 字典空间为 $(1 + o(1)) \cdot n' \cdot (\log(2^{\ell_i}/2^i) + r)$ 。代入 $\ell_i - i = \log(1/\varepsilon) + O(1)$ 及 $r = \lceil \log \log u \rceil = \log \log n + O(1)$ ，得：

$$|D_i|_{\text{space}} = (1 + o(1))n' \log(1/\varepsilon) + O(n' \log \log n)$$

由于桶化优化，该空间没有常系数的冗余，全局空间匹配下界。

6.3.8 去摊销化：最坏情况常数时间插入

不再等到子序列第一个元素时才一次性初始化 D_i ，而是将初始化 D_i 的工作均匀分散到该子序列的 2^i 次插入操作中。每次插入额外投入常数步工作于 D_i 的渐进构建，使得插入变为**最坏情况** $O(1)$ （高概率）。

代价：不能使用桶化优化——多个桶可能同时过渡，无法有效平摊。空间从 $(1+o(1))n \log(1/\varepsilon) + O(n \log \log n)$ 升至 $O(n \log(1/\varepsilon) + n \log \log n)$ ，前导常数略有增加，但渐近阶不变。

6.3.9 支持删除操作（扩展）

近似成员数据结构无法检测用户是否试图删除假阳性元素，因此要求用户保证“只删除确实在集合中的元素”。论文的方案是维护两个字典：

- **主字典**：同原存储结构，但每个条目多一个删除指示位。
- **辅助字典**：存储“发现假阳性”的元素的精确值（零假阳性率）。

插入时先检查当前元素是否是假阳性（即插入流中不重复），若是则放入辅助字典，否则正常插入主字典。删除时先查辅助字典，有则删除；没有则查主字典，找到“最小字典序条目”并标记为已删除。后台进程定期清理已删除签名的空间。

7 总结与展望

7.1 上下界对比与紧致性

论文的核心结论是：在未知集合大小 n 的前提下，近似成员查询问题的空间复杂度存在上下界的高度匹配：

- **下界 (定理 1.1):** $\Omega(n \log(1/\varepsilon) + n \log \log n)$
- **上界 (定理 1.2):** $O(n \log(1/\varepsilon) + n \log \log n)$

两者渐近阶完全一致，说明未知规模带来的 $\log \log n$ 的平均开销是**不可避免的固有代价**。换言之，不存在渐近更优的数据结构。

7.2 $\log \log n$ 的意义

1. **理论与实践的分离：**证明了已知 n 和未知 n 这两种模型确实存在超线性空间差距。已知 n 时每元素开销为常数 $\Theta(\log(1/\varepsilon))$ ，未知 n 时多出了随 n 增长的 $\Omega(\log \log n)$ 项。
2. **增长极为缓慢：** $\log \log n$ 本身增长极其缓慢，相比旧方案（如 Scalable Bloom Filter）的 $\Omega(\log n)$ 开销更具优势。例如当 $n = 2^{64}$ 时， $\log \log n$ 仅为 6——在实际中几乎可以视为常数。
3. **理论开创性：**首次给出了自适应扩容结构的固有理论存储下界，为工程实现提供了严格的理论证明和设计指导。

7.3 过往方案为何不优

在过往的方案中（如 Scalable Bloom Filter），为了保证总假阳性 $\leq \varepsilon$ ，每一级子过滤器的假阳性 ε_i 需要按 $1/i^2$ 衰减，每级都要存储独立的哈希函数，累加后总空间开销达到 $\Omega(n \log n)$ ，远非最优。本文的构造 1 虽同样采用多级结构，但通过精心分配 $\varepsilon_i = \Theta(\varepsilon/i^2)$ 、使用空间近最优的 Raman-Rao 字典作为底层，成功将空间降至 $O(n \log \log n)$ 的附加项。构造 2 更进一步，通过单字典 + 签名扩展 + 辅助位技术，在维持同等空间的同时实现了常数时间查询。

7.4 未来研究方向

- **理论方向：**收紧 $\Omega(n \log \log n)$ 附加项的常数因子，探索是否能在不牺牲渐近界的前提下进一步降低隐藏常数。

- **实践方向：**构造 1 查询时间为 $O(\log n)$ 较慢，构造 2 隐藏常数较大。期待设计同时满足最优空间和常数时间的实用数据结构，真正匹配本文的空间下界。
- **工程优化：**论文作者明确指出后续实际实现仍需进一步优化隐藏常数和工程细节，包括去摊销版本的空间常数优化、桶化的实际性能调优等。